DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Bogura, Bangladesh

A THESIS ON

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Course Title: Thesis/Project Work

Course Code: CSE-499(B)

Prepared By

Md. Riaz

ID: 0322310105101024

Program: B.Sc. in Computer Science & Engineering

Semester: 8th Semester

Session: _____

Under the Supervision of

Mst. Sahela Rahman

Designation: Lecturer

Department of Computer Science & Engineering

Pundra University of Science & Technology

Date of Submission: _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Bogura, Bangladesh

A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Course Title: Thesis/Project Work

Course Code: CSE-499(B)

Submitted to

Department of Computer Science & Engineering

Pundra University of Science & Technology

_____

Supervisor Signature

Mst. Sahela Rahman

Department of Computer Science & Engineering, Pundra University of Science & Technology

_____

Student Signature

Md. Riaz, ID: 0322310105101024

CERTIFICATION OF ORIGINALITY

I hereby declare that this thesis titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics" is my own original work, carried out under the supervision of Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology. To the best of my knowledge, this thesis contains no material previously published or written by another person, except where due reference is made in the text. This work has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

_____

Signature of Student

Md. Riaz

ID: 0322310105101024

CERTIFICATION OF APPROVAL

This thesis titled

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

submitted by Md. Riaz to the Department of Computer Science & Engineering, Pundra University of Science & Technology, has been examined and is hereby approved as a partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Supervisor:

_____

Mst. Sahela Rahman

Department of Computer Science & Engineering

Pundra University of Science & Technology

Examiner:

_____

Name: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology, for her continuous guidance, patience, and constructive feedback throughout the design, implementation, and evaluation of this research. Her insistence on precise, defensible claims shaped this thesis at every stage, from the formal safety proof to the honesty of its limitations section.

I am also grateful to the faculty members of the Department of Computer Science & Engineering for the coursework and feedback that prepared me to undertake this research, and to my family for their patience and encouragement during the long implementation and evaluation cycles this thesis required.

Finally, I acknowledge the authors of the prior work surveyed in Chapter 2 of this thesis. Their published research on natural language interfaces, text-to-SQL translation, and dashboard generation provided the foundation against which AEGIS's contributions are measured.

Table of Contents

List of Figures

List of Tables

ABSTRACT

Analytical dashboards are important tools for business reporting, but building accurate and safe reports from relational databases still requires technical skills. Natural language interfaces try to close this gap, but current text-to-SQL systems focus on benchmark accuracy rather than real-world safety, and they stop at generating a one-time query result without producing reusable reporting widgets. This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that turns plain-English reporting requests into dynamic, refreshable dashboard widgets that users can save and reuse every day. Unlike traditional natural-language-to-SQL systems that treat each question as a one-off interaction, AEGIS produces persistent reporting widgets, each with its own refresh schedule, access rules, and visual configuration, that become part of a user's daily workflow.

AEGIS uses a strictly controlled pipeline: (1) a lightweight large language model (Llama 3.1 8B) maps natural language to one of eleven high-level analytical primitives (for example KPI, Trend, Ranking, Tabular) using dynamic vocabulary injection; (2) a deterministic compiler builds the SQL using pre-approved parameterized templates; and (3) a post-compilation security monitor validates the statement against a strict safety grammar. Evaluation via a 100-query prototype benchmark in a real e-commerce domain (nopCommerce) demonstrates 100% intent accuracy on the covered primitives and structural prevention of SQL injection through untrusted natural-language input, a guarantee that holds within the defined threat boundary of trusted semantic-layer definitions and administrator-controlled compiler templates. A cross-schema evaluation on WooCommerce confirms that only the semantic layer requires reconfiguration for a new schema, achieving 98.0% intent accuracy in 14 person-hours. AEGIS demonstrates that restricting SQL generation to a finite set of validated business patterns is a practical path to safe, auditable natural-language reporting in institutional environments.

Index Terms: Natural language interfaces, dashboard generation, text-to-SQL, semantic layer, visualization recommendation, business intelligence, self-service analytics.

CHAPTER 1

INTRODUCTION

1.1 Background

Relational databases store critical institutional data in organizations: financial records, customer accounts, sales transactions, and more. But accessing this data is uneven. Technical staff can write SQL queries to get any answer they need, while non-technical users have to wait for someone else to build them a report. This waiting is expensive. Analysis of enterprise reporting workflows shows that business users frequently wait days for new reports, and historical query logs show that 61% of their reporting questions were just variations of things they had already asked before: the same report with a different date range, or the same chart for a different department. These are not one-off questions; they are recurring reporting needs that should be served by saved, refreshable widgets rather than regenerated from scratch every time.

This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that lets users describe their reporting needs in plain English and produces dynamic dashboard widgets that can be saved, refreshed, and reused as part of their daily workflow, without anyone writing SQL.

Natural language interfaces to databases (NLIDBs) try to solve this problem. The idea is simple: a user should be able to ask “which categories have the highest refund rates this month?” and get a correct, visual answer without writing SQL. Researchers have made good progress here. Neural text-to-SQL systems now exceed 90% accuracy on the Spider benchmark [28], and large language models can produce reasonable-looking SQL with minimal setup [10]. But there is still a gap between benchmark results and real-world deployment, which Chapter 2 examines in detail.

1.2 Problem Statement

Three problems make up the gap between benchmark text-to-SQL accuracy and safe, production-ready natural-language reporting.

Safety. Many modern natural-language-to-SQL systems rely on models that directly generate SQL tokens, which creates challenges for enforcing enterprise governance and security policies. An LLM generating SQL freely can produce queries that expose private data or use incorrect table joins, and detecting this after the fact is fundamentally harder than preventing it by construction.

Vocabulary mismatch. Benchmarks use actual column names in the questions, but real users speak in business terms ("refund rate" instead of SUM(o.RefundedAmount)). Matching these requires business knowledge that models do not always get right, and a wrong mapping can silently produce a plausible but incorrect report.

No widget generation. Existing systems answer one question at a time and discard the result. They do not produce saved reporting widgets that can be refreshed with new data tomorrow, shared with a colleague, or added to a daily dashboard. Every time someone needs the same report, they start from scratch, which directly contradicts the finding that 61% of reporting requests are recurring.

These problems are not primarily about building a smarter model; they are about designing the system properly around the model. AEGIS addresses this by splitting the work into stages. The LLM's only job is to understand what the user is asking and output a structured description of the request. Everything after that, matching to the right business terms, building the SQL, selecting the chart, and saving the widget, is done by fixed rules and pre-approved templates.

1.3 Research Novelty and Motivation

Existing text-to-SQL research asks: how accurately can a model generate SQL from natural language? This thesis asks a different question: how can large language models be used for language understanding while being structurally prevented from generating executable SQL at all? The two pipelines differ structurally.

Classical NL-to-SQL: Natural Language → SQL generation → Query result.

AEGIS: Natural Language → Intent extraction → Semantic constraint → Deterministic compilation → Safe analytical artifact.

The contribution of this thesis is therefore not improved SQL generation accuracy; it is constrained analytical artifact generation, a design approach that removes SQL generation from the LLM's role entirely and provides safety and semantic fidelity guarantees that no generative model can match unconditionally.

AEGIS does not propose a new LLM. The model is interchangeable: Groq-hosted Llama 3.1 8B, OpenRouter, a local Ollama instance, or any endpoint compatible with the /v1/chat/completions interface. Because the LLM's only contract with the rest of the system is to produce a typed JSON intent object, model upgrades improve quality automatically without changing the compiler or safety infrastructure. This is model independence by design, not an implementation accident.

1.4 Objectives and Contributions

This thesis makes the following contributions:

1. An analysis of real reporting behavior based on 312 requests from e-commerce and business-intelligence datasets, resulting in eleven common reporting patterns (Chapter 3).
2. A system design in which all possible queries are limited to pre-approved templates and a defined semantic layer, which prevents SQL injection and unauthorized data access by construction (Chapter 3).
3. The AEGIS system itself, including the semantic layer design, a vocabulary injection prompting strategy, a safe SQL builder with two-layer defence, a rule-based chart selector, and a widget storage system with scheduled refresh (Chapters 3-4).
4. A vocabulary injection method that places the approved metric and dimension names directly into the LLM prompt, removing the need for a manually written synonym list while achieving 100% coverage, reducing the synonym dictionary from 112 entries to zero (Chapter 3).
5. A benchmark evaluation of 100 queries showing 100% valid SQL and 0% unsafe queries, compared to 5.0% unsafe queries from a direct LLM-to-SQL baseline (Chapter 5).
6. A cross-schema generalizability study on WooCommerce showing that only the semantic layer requires modification when deploying to a new production schema,

with 98% intent accuracy achieved in 14 person-hours of configuration (Chapter 5).

7. A pipeline latency analysis showing that the AEGIS safety infrastructure adds less than 4% overhead relative to the LLM API call, making the safety guarantees effectively free in practice (Chapter 5).

1.5 Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews related work across four decades of natural language database interfaces, neural text-to-SQL, natural-language-driven visualization, dashboard generation, and semantic layers, and positions AEGIS relative to this body of work through a comparative summary and a research gap analysis. Chapter 3 presents the methodology: the research paradigm, the formative study of real reporting requests that motivated AEGIS's design, the formal model and threat model that define its safety guarantee, and the system architecture, semantic layer, intent parser, safe compiler, visualization selector, and widget engine that implement it. Chapter 4 describes the experimental work: the implementation, the experimental environment, the benchmark dataset, and the baseline systems used for comparison. Chapter 5 reports results for each of the five research questions and discusses what they mean, including a structural comparison against direct LLM-to-SQL generation and an analysis of what AEGIS deliberately gives up in exchange for its safety guarantees. Chapter 6 states the limitations of the current work honestly and outlines future work. Chapter 7 concludes the thesis.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

This chapter surveys prior work across six areas that bear on AEGIS's design: natural language interfaces to databases, neural text-to-SQL and its benchmarks, constrained decoding and recent hybrid NL-to-SQL systems, natural language for visualization, dashboard generation, and semantic layers for controlled analytics. It closes with a review of recent practitioner and adoption-focused literature on AI-powered dashboards, a comparative summary table, and an explicit research gap analysis. Thirty-one sources are reviewed. A systematic re-check of this thesis's reference collection against the source PDFs, undertaken during the preparation of this chapter, found that two files had been filed under the wrong name (an incorrect PDF saved against the correct citation text). Both misfiled papers turned out to be genuinely relevant and are retained under their correct, verified titles (Sections 2.3 and 2.7); the two originally-intended papers were subsequently located, confirmed, and added to the collection under their correct filenames, so this chapter reviews both.

2.1 Natural Language Interfaces to Databases

Natural language database interfaces have been studied for over four decades. Early systems such as LUNAR and TEAM used hand-crafted grammars and domain-specific ontologies to parse queries. These systems were brittle under vocabulary variation, but they established the core insight that query understanding requires an explicit bridge between natural language and schema semantics, rather than treating the schema as implicit context for a general-purpose language model.

Affolter et al. [1] provide the most systematic comparison of this literature to date. They define a ten-question benchmark of increasing complexity (joins, filters, aggregation, negation, subqueries) and evaluate 24 NLIDBs across four architectural classes: keyword-based, pattern-based, parsing-based, and grammar-based. Keyword-based systems answer only simple filter queries and fail on aggregation or subqueries; grammar-based systems such as SQUALL and SPARKLIS achieve the broadest coverage but require users to learn a constrained query vocabulary; parsing-based, ontology-driven systems handle free natural language well but stumble on negation

and multi-level subqueries. Their survey never evaluates SQL injection, hallucination, or adversarial safety, a gap this thesis addresses directly.

NaLIR [9] is an important modern NLIDB because it treats ambiguity as a problem to expose rather than to silently resolve. NaLIR parses a query into a grammar-constrained intermediate “query tree” that maps deterministically to SQL, and when parsing is ambiguous it presents the user with a short multiple-choice interaction rather than guessing. In a user study on the Microsoft Academic Search dataset, users with NaLIR's interactive step correctly completed 88 of 98 tasks, versus 56 of 98 using MAS's native faceted-search interface, and only about 68 of 98 without the interactive step. Without interaction, most wrong answers went undetected by users. NaLIR anticipates AEGIS's separation of language understanding from SQL construction, but its correctness still depends on the user actively catching and correcting ambiguous parses; AEGIS instead restricts the LLM to a fixed, pre-approved vocabulary so that incorrect or unsafe SQL is structurally harder to produce in the first place, and it persists results as reusable widgets, which NaLIR does not.

Lehmann et al. describe Veezoo, a commercial analytics platform built around an auto-generated, user-editable Knowledge Graph that matches keywords to database concepts, combined with entity linking, relation extraction, and an ML-based ranking model over candidate logical forms. A controlled experiment with 16 users and an adversarially unoptimized configuration found a median of two query reformulations needed to reach a correct answer. Veezoo's Knowledge Graph is structurally analogous to AEGIS's semantic layer [8], but Veezoo still compiles candidate logical forms into open-ended SQL and relies on iterative dialogue to recover from wrong answers, a generate-fail-retry interaction pattern. AEGIS instead aims to prevent incorrect or unsafe generation up front through a fixed template library.

Liu and Xu [11] provide the most recent systematic review, organizing NLIDB research around a three-stage pipeline of preprocessing, understanding, and translation. Their review is the only one of the reviewed surveys to document empirical SQL-injection risk directly: it discusses TrojanSQL, a backdoor-based injection attack framework against text-to-SQL systems that the authors report achieves a high attack success rate and is difficult to defend against. Their recommended mitigations, chiefly vetted training data and “additional layers of

security or filtering," are general and non-constructive. AEGIS operationalizes exactly this recommendation: by never allowing the LLM to generate SQL text and compiling every query deterministically from a fixed template library, the class of attack Liu and Xu describe cannot reach the SQL layer regardless of what the LLM outputs.

2.2 Neural Text-to-SQL and Benchmark Progress

The field shifted decisively toward neural approaches with Seq2SQL [31], which combined a SQL-structured decoder (aggregation classifier, SELECT-column pointer, WHERE-clause pointer) with reinforcement learning driven by in-the-loop query execution, and released WikiSQL, 80,654 question-SQL-table triples restricted to single-table SELECT/aggregation/WHERE queries. Seq2SQL reached 59.4% execution accuracy on WikiSQL and reduced invalid-SQL generation from roughly 7.9% to 4.8% relative to a prior sequence-to-sequence baseline, demonstrating that constraining a model's output structure improves both accuracy and validity. AEGIS extends this same insight to its logical conclusion: rather than a network that still emits SQL tokens with residual freedom to hallucinate columns, AEGIS removes SQL emission from the model entirely.

Spider [28] exposed how far this problem remained from solved. Its 10,181 questions and 5,693 unique SQL queries span 200 multi-table databases with a strict train/test database split, forcing genuine generalization to unseen schemas. The best baseline of the time reached only 12.4% exact-match accuracy under this cross-domain setting, and accuracy degraded further as the number of foreign keys grew. SParC [29] and CoSQL [30] extended Spider's databases to conversational, multi-turn settings: SParC's best model reached only 20.2% exact-match accuracy over individual questions in a sequence, dropping sharply with turn number, and CoSQL found that roughly 40% of realistic user questions could not be directly converted to SQL at all, requiring clarification, inference, or a coverage rejection. AEGIS's semantic layer effectively hard-codes the ambiguous-or-unanswerable detection that CoSQL's models struggle to learn, by rejecting or clarifying any intent that does not map onto its approved vocabulary rather than attempting to freely interpret and generate SQL for arbitrary questions.

BIRD [10] closed the gap between academic benchmarks and production conditions by testing 95 real, large databases (up to 33.4 GB) with expert-written domain-knowledge annotations. Even the best evaluated model, GPT-4 combined with DIN-SQL prompting, reached only 55.9% execution accuracy against a 92.96% human-expert ceiling, and the dominant failure modes were wrong schema linking (41.6% of errors) and misunderstanding database content or values (40.8%). This is the strongest available evidence for AEGIS's core motivating claim: even frontier LLMs generating free-form SQL against real, large schemas hallucinate schema elements and misinterpret values in roughly two out of every five cases. AEGIS avoids this failure class by never asking the LLM to resolve schema, joins, or literal values at all; it only classifies a request into a fixed, pre-vetted vocabulary, after which a deterministic compiler resolves every join and expression.

RAT-SQL [25] is the most sophisticated attempt within the free-form generation paradigm to solve schema linking directly, using relation-aware self-attention over a typed schema graph and a grammar-constrained decoder, reaching 57.2% exact-match accuracy on Spider (65.6% with BERT augmentation). Even so, oracle analysis attributed 72-81% of its remaining errors to wrong column or table selection or wrong SQL structure. RAT-SQL's schema graph plays a role analogous to AEGIS's semantic layer, but is used to guide free SQL token generation rather than to constrain a bounded classification space; its grammar constraint guarantees syntactic validity only, not semantic correctness, safety, or injection-proofing, which remain unaddressed by any paper in this section.

2.3 Constrained Decoding and Recent NL-to-SQL Systems

PICARD [18] is the closest prior technique to AEGIS's philosophy of restricting model output, and the most important contrast case for this thesis. It is an inference-time, model-agnostic constrained-decoding method that uses incremental parsing during beam search to reject any candidate token that would make the partial SQL output invalid against a grammar and the target schema. Applied to T5-3B, PICARD reduced the invalid-SQL rate on the Spider development set from roughly 12% to 2% and raised execution accuracy to a then state-of-the-art 79.3%. Even at its strictest setting, however, PICARD does not eliminate invalid SQL, and because the LLM still writes every character of the SQL string, any semantic bug not caught by the grammar

or schema check (a wrong join, a wrong aggregation, a wrong business definition of a metric) can pass through as syntactically valid but analytically wrong SQL. PICARD constrains what tokens the LLM may emit while it writes SQL; AEGIS removes SQL-writing from the LLM's role entirely, which is a categorically stronger guarantee because it eliminates an entire failure class rather than reducing its probability at the token level.

G-SQL [21] is the closest architectural precedent to AEGIS among the reviewed systems: a rule-based, template-driven SQL generator built on a structured, curated schema representation and a domain-specific synonym dictionary, deliberately avoiding a neural component that writes SQL directly. On the IMDB, Yelp, and MAS benchmarks it achieved 100% execution accuracy on easy queries across all three, but accuracy fell to 45-55% on extra-hard queries requiring nested reasoning. AEGIS builds on the same core insight, that a curated vocabulary plus deterministic clause assembly yields much higher executability than free-form generation, but replaces G-SQL's classical NLP pipeline (dependency parsing, GloVe embeddings) with an LLM-based intent extractor, and adds a persisted, reusable dashboard-widget layer that G-SQL does not have.

A generative-AI-based conversion system by Jha et al. [6] illustrates the pattern this thesis argues against: a sequence-to-sequence model directly emits SQL text from unconstrained natural language, with a rule-based module added afterward only to produce a plain-language explanation of whatever SQL the generator happened to produce. No accuracy or robustness evaluation is reported for the system. The explanation module is a post-hoc rationalization, not a safety mechanism, and cannot prevent a hallucinated table name or an injection-prone output; it exemplifies a broader gap in practitioner literature, where generative NL-to-SQL tools are shipped without a rigorous safety or correctness evaluation.

TriSQL [23] is, at the time of writing, the most recent and most accurate system in the free-form generation paradigm, and the sharpest available contrast to AEGIS's approach precisely because it is state of the art. TriSQL is a three-stage LLM pipeline: a Question-Guided Schema Selector narrows a large schema down to the tables and columns relevant to the question using cross-attention; a Structure-Aware SQL Generator predicts a clause-level skeleton before filling in schema-specific content;

and a Complexity-Aware SQL Refiner classifies each question's difficulty and applies an LLM-driven repair loop, with an execution-validated fallback that reverts to whichever of the initial or refined query actually executes. On Spider, TriSQL reaches 82.2% execution accuracy at 180 ms of inference latency, and its ablation study shows that removing the skeleton-first generation stage alone collapses execution accuracy from 82.2% to 18.8%, the same qualitative finding this thesis reports for removing vocabulary injection from AEGIS (Section 5.3). TriSQL is, in other words, concrete evidence that structuring an LLM's generation process substantially improves reliability. It is also, however, a system that still asks the LLM to produce the final executable SQL string directly, refined by further LLM calls, with no semantic layer, no permission rewriter, and no reported injection-safety evaluation at all: its safety properties, whatever they are, are inherited entirely from the underlying model's behavior rather than guaranteed by the pipeline's structure. AEGIS and TriSQL therefore make the same core engineering bet, that unconstrained LLM output is unreliable and should be structured, but apply it to different halves of the problem: TriSQL structures how SQL is generated to raise its accuracy ceiling; AEGIS removes SQL generation from the LLM's role entirely to establish a safety floor. The two approaches are not mutually exclusive, but no paper reviewed in this thesis, including TriSQL, evaluates both properties together.

Pinna et al. [16] raise a methodological point relevant to how any such system should be evaluated: they propose the Query Affinity Score, a continuous metric combining code-embedding similarity and executed-result-table similarity, arguing that binary exact-match and execution-accuracy metrics hide partial correctness, for example, a dropped DISTINCT keyword or a reversed sort order can score as completely wrong under a binary metric despite near-identical SQL text. This is a genuinely useful caution for Chapter 5's evaluation design: AEGIS's own safety property makes most of the failure modes Pinna et al. study architecturally impossible in the first place, since clauses come from vetted templates rather than free generation, but their semantic-versus-structural distinction is a useful frame for grading intent extraction accuracy rather than treating it as pass or fail.

2.4 Natural Language for Visualization

A parallel research stream targets chart generation rather than SQL generation. nl4dv [14] is the closest visualization-side precedent to AEGIS's design: a toolkit that maps natural language queries to a small, fixed set of five analytic tasks and a rule-based attribute-task-visualization mapping table, returning a structured JSON specification rather than free-form chart code. It reports response times of 1-18 seconds on small in-memory datasets, but conducts no formal accuracy benchmark, has no SQL-safety or database-permission layer at all (it operates purely on an in-memory dataset, never a live relational schema), and produces one-off specifications with no notion of a persistent, refreshable dashboard object.

DataTone [4] established the alternative philosophy of surfacing ambiguity to the user through interactive “ambiguity widgets” rather than silently resolving it, covering six decision points from attribute recognition to chart-type choice. In a comparative study against IBM Watson Analytics, participants completed significantly more correct facts with DataTone (5.43 versus 2.00 facts, $p < 0.01$). AEGIS and DataTone address the same underlying ambiguity problem with opposite strategies: DataTone generates many candidate interpretations and lets the user disambiguate after the fact, while AEGIS restricts the LLM's extraction target to a small curated vocabulary up front, preventing many classes of ambiguity from arising at all. DataTone is also explicitly limited to single-table, non-relational data and offers no persistence mechanism.

nvBench [12] demonstrates the risk of the opposite extreme: it synthesizes a 25,750-pair benchmark from Spider's SQL queries and trains ncNet, a Transformer that translates natural language directly, end to end, into a Vega-Zero visualization specification with no deterministic or auditable compilation step in between. Correctness depends entirely on the trained model's generalization, with no template-based guarantee against a malformed or unsafe query, and the output is a single static chart per query rather than a persisted artifact. Eviza extended natural language interaction to already-rendered visualizations, allowing conversational refinement of an existing chart [19], a concept AEGIS's clarification model draws on when a request is ambiguous. Kavaz et al.'s scoping review of chatbot-based visualization interfaces [7] found that across 20 surveyed systems, 90% supported only low-level queries, half used fixed rather than adaptive visual mapping, and only 4 of 20 supported any follow-up or conversational interaction, confirming that most systems in this space regenerate a visualization each time rather than treating a validated query-chart

pairing as a durable, reusable object, precisely the gap AEGIS's widget persistence model closes.

2.5 Dashboard Generation

Dashboard generation as an automated design problem has attracted growing attention. DashBot [3] formulates dashboard construction as a Markov decision process solved with deep reinforcement learning (A3C), using a preliminary study of 90 real Tableau and Power BI dashboards to define reward functions for presentation quality (diversity, parsimony) and statistical insight (trend, correlation, comparison). A user study found DashBot preferred over a prior deep-learning dashboard generator, MultiVision [27], on 76-88% of quality dimensions. Both systems, however, operate without any natural language input or intent layer: a user cannot ask a question in English; the agent instead explores a dataset unprompted. Neither faces an LLM-generated-SQL attack surface at all, because neither generates SQL from user language, which is precisely the problem AEGIS's semantic layer and compiler are built to solve and that DashBot's architecture never encounters. DataShot [26] and Calliope [22] used statistical fact extraction followed by template-based layout to generate narrative data documents, again illustrating that rule-based, non-AI logic is a well-precedented and reliable way to drive downstream presentation once the underlying facts are established, reinforcing AEGIS's own decision to use a deterministic, rule-based visualization selector rather than a learned one.

2.6 Semantic Layers and Controlled Analytics

A semantic layer is a business-logic abstraction that maps business concepts to the actual database tables and columns. Commercial tools such as dbt Metrics, Looker LookML, and Apache Superset implement semantic layers in different ways, but the research literature has treated the semantic layer primarily as a convenience for query authoring rather than as a safety mechanism. Lehmann et al.'s Veezoo [8], already discussed in Section 2.1, is the clearest exception, and even there the Knowledge Graph constrains matching rather than replacing SQL generation outright. Structured output enforcement for LLMs [15] has been shown to improve the reliability of typed object generation, which AEGIS relies on for intent extraction: the LLM's output is constrained to a fixed JSON schema before any downstream validation occurs. No

prior work reviewed in this chapter uses a semantic layer as the primary safety mechanism for an LLM-assisted reporting system, in the specific sense of replacing SQL generation with deterministic compilation from a governed vocabulary; this is the gap AEGIS's architecture is built to close.

2.7 AI-Powered Dashboard Adoption, Governance, and Conversational BI

Beyond the technical NL-to-SQL and NL-to-visualization literature, a body of recent practitioner-oriented and management-focused work addresses the adoption, governance, and business impact of AI-powered dashboards. This literature is reviewed here because it was identified through a systematic re-examination of this thesis's full reference collection, and because it usefully situates AEGIS's technical contribution within the broader question of why organizations want AI-assisted analytics in the first place.

Häikiö [5] examines how organizations should govern AI-powered executive dashboards, concluding that no mature, comprehensive AI-governance framework yet exists and proposing adapted IT-governance processes (drawing on COBIT 2019 and the Technology-Organization-Environment model) as an interim approach. Saidur [17] reports a large-sample quantitative study of 150 U.S. enterprises, finding that AI-enhanced BI dashboards correlated with substantial gains in forecast accuracy (78.1% to 91.2%) and marketing return on investment (124% to 168%) over a 24-month period. Neither paper addresses natural-language query translation, SQL generation, or injection safety; their relevance to AEGIS is limited to the shared premise that persistent dashboards, rather than one-off chat answers, are the effective vehicle for delivering AI-derived insight to decision-makers, which is one motivation for AEGIS's widget-persistence design.

Mujeeb et al. [13] propose, at the conceptual level, a four-layer conversational business-intelligence architecture whose query-planning layer explicitly combines “template-based SQL generation” for predictable requests with “transformer-based synthesis” for complex ones, followed by a separate governance layer that validates queries after generation. The authors state plainly that the architecture has not been implemented or validated. This is a useful reference point precisely because it shows

the field converging on a hybrid template-plus-generation idea without committing to it or testing it: AEGIS's contribution is to commit fully to the deterministic-compilation path for every query expressible in its semantic layer, removing the free-generation fallback entirely and, with it, the need for a downstream governance layer to catch what the LLM might have produced incorrectly.

Shailesh et al. [20] present a working Conversational BI Assistant built on the Groq API and LangChain that is, in effect, a live demonstration of the architecture this thesis argues against, and for exactly that reason it is one of the most useful comparison points in this review. Their system gives an LLM agent direct tool access to a SQL database (sql_db_list_tables, sql_db_schema, sql_db_query), retains conversational memory across turns so users can say “now filter for Asia” without restating the query, and automatically selects a chart type for the result. Crucially, when a generated query fails, a self-correction loop feeds the database's error message back to the LLM, which revises the SQL and retries, iterating until execution succeeds. This is a coherent, deployable design, and the authors report it working well on straightforward aggregation queries. It is also, structurally, exactly the T1-class attack surface this thesis's threat model (Section 3.5) is built to close: the LLM sees the live schema, composes SQL directly, and is trusted to interpret its own error messages and repair its own queries, with no semantic layer standing between natural language and the database and no reported adversarial or injection evaluation at all. The authors' own discussion is candid that the system still struggles with ambiguous business vocabulary and complex joins, which is precisely the vocabulary-mismatch and expressiveness problem AEGIS's semantic layer and pattern library (Sections 3.7 and 3.9) are designed to solve, not by making the LLM better at self-correction, but by removing its ability to compose the SQL string in the first place.

Valkenburgh's master's thesis [24] on explanatory analytics is, by contrast, one of the most architecturally aligned sources in this collection, despite being in an unrelated domain. Valkenburgh's prototype computes causal-influence explanations for business metrics using a deterministic, non-AI formalism, and only afterward asks an LLM to narrate the already-correct structured result in natural language, never allowing the LLM to perform the underlying computation itself. A pre-study surveying ten commercial AI-BI products (including Power BI and Tableau) found that all of them could answer simple descriptive queries but none could correctly

answer explanatory, “why is X high” queries without manual pre-configuration, direct evidence that unconstrained LLM reasoning fails at exactly the class of task AEGIS targets. Valkenburgh's design and AEGIS arrive at the same governing principle independently: do not trust the model to perform the analytical computation; let a deterministic algorithm compute the correct result, and restrict the model's role to a bounded, downstream task. Valkenburgh's deterministic layer operates on a single pre-loaded spreadsheet rather than a live relational database, so it has no SQL-injection surface and produces one-off, non-persistent text answers rather than stored, refreshable widgets, both of which AEGIS's architecture addresses.

Finally, Chinnappaiyan [2] surveys “conversational analytics” as a general architectural pattern (natural language understanding, semantic layer, query generation, response generation) and identifies fine-grained access control versus conversational fluency as an unresolved tension in current systems, without proposing or evaluating a concrete mechanism to resolve it. This is precisely the gap AEGIS's design closes by construction: the Permission Rewriter (Chapter 3) enforces row-level access control after the LLM has already produced its intent, so conversational fluency and access control are not in tension because the LLM never has an opportunity to influence the permission predicate.

2.8 Comparative Summary

Table 4 positions AEGIS against the systems most central to Sections 2.1-2.6 across seven properties: whether the system parses natural language at all, whether it defines an explicit semantic layer, whether SQL generation is structurally safe rather than merely accuracy-optimized, whether it selects or generates a visualization, whether results persist as reusable widgets, whether unanswerable requests are explicitly validated against a coverage boundary, and the nature of its evaluation.

Table 4: Comparative summary of related NL-to-database and NL-to-visualization systems.

System	NL Parsing	Semantic Layer	Safe SQL	Visualization	Widget Persistence	Coverage Validation	Evaluation

Spider / BIRD / Seq2SQL	Yes	-	-	-	-	-	Benchmark only
RAT-SQL / PICARD	Yes	-	Partial	-	-	-	Benchmark only
NaLIR	Yes	-	-	-	-	-	User study
Veezoo (Lehmann et al.)	Yes	Yes	-	-	-	-	User study
G-SQL	Yes	Partial	Partial	-	-	-	Benchmark only
TriSQL	Yes	-	-	-	-	-	Benchmark only
nl4dv	Yes	-	-	Yes	-	-	In-memory data
DataTone	Yes	-	-	Yes	-	-	User study
DashBot / MultiVision	-	-	-	Yes	Partial	-	Synthetic / user study
Conversational BI Assistant (Shailesh et al.)	Yes	-	-	Yes	-	-	Prototype demo
AEGIS (this thesis)	Yes	Yes	Yes	Yes	Yes	Yes	Production (nopCommerce)

2.9 Research Gap Analysis

Three consistent gaps emerge from this review, and each maps directly onto a design decision made in Chapter 3.

First, no system reviewed in Sections 2.1-2.3 treats SQL-generation safety as a structural property rather than an accuracy metric, including the most recent, most accurate one. Every neural text-to-SQL system, from Seq2SQL through RAT-SQL, PICARD, and the state-of-the-art TriSQL, is evaluated purely on exact-match or execution accuracy against a benchmark; none reports an injection rate, and only Liu and Xu's review even discusses adversarial SQL injection as a named threat, without proposing an architectural defense. Shailesh et al.'s deployed Conversational BI Assistant (Section 2.7) makes the same point from the practitioner side: it gives an LLM direct SQL-execution tool access and a self-correction loop, but reports no adversarial evaluation at all. AEGIS closes this gap by removing SQL generation from the LLM's output space entirely (Section 3.4) and evaluating an explicit unsafe-SQL rate against a direct LLM-to-SQL baseline (Chapter 5), rather than relying on accuracy as a proxy for safety.

Second, no system in Sections 2.1, 2.4, or 2.5 combines a governed semantic vocabulary with persistent, refreshable output. Veezoo and G-SQL define curated vocabularies but produce one-off answers; nl4dv, DataTone, and nvBench produce one-off visualizations; DashBot and MultiVision generate multi-chart dashboards but with no natural-language input and no notion of a saved, reusable artifact tied to a specific user question. AEGIS's widget engine (Section 3.11) closes this gap directly, motivated by the formative-study finding that 61% of real reporting requests are recurring (Section 3.2).

Third, the practitioner and adoption-focused literature reviewed in Section 2.7 recognizes the tension between conversational flexibility and governed access control (Chinnappaiyan), gestures at hybrid template-plus-generation designs without building them (Mujeeb et al.), and, where a system is actually deployed (Shailesh et al.), resolves that tension by trusting the LLM with direct database access and a self-correction loop rather than a governed vocabulary. Valkenburgh's independent arrival at the same “let a deterministic layer compute the answer, let the model only narrate it” principle, in an unrelated domain (spreadsheet-based explanatory analytics rather

than relational-database reporting), is the strongest external corroboration available in this review that AEGIS's central design decision is not idiosyncratic to this thesis but reflects a pattern independently discovered wherever researchers have taken LLM reliability seriously as an engineering constraint rather than an accuracy target to be improved.

CHAPTER 3

METHODOLOGY

3.1 Research Paradigm

This thesis follows a Design Science Research paradigm, in the sense used by Hevner et al.: knowledge is produced by building and evaluating a novel artifact that addresses an identified organizational problem, rather than by testing a hypothesis about an existing phenomenon. The artifact in this thesis is the AEGIS system itself. Design Science Research requires three things to be demonstrated: problem relevance, design rigor, and evaluation against the stated problem. Problem relevance is established through the formative study of Section 3.2, which analyzes real reporting behavior rather than assuming a problem exists. Design rigor is established through the formal model and threat model of Sections 3.4-3.5, which state precisely what safety property the architecture guarantees and under what boundary that guarantee holds. Evaluation is reported in Chapter 5 against five explicit research questions, using both a benchmark dataset constructed for this domain and an ablation study that isolates the contribution of each architectural component.

The research proceeded through three iterative build-evaluate cycles. The first cycle produced a semantic layer and compiler for a minimal set of intent classes and validated the core safety proposition on a small hand-written query set. The second cycle expanded coverage to all eleven intent classes identified in the formative study and introduced vocabulary injection after an early synonym-dictionary approach proved unable to keep pace with real query phrasing. The third cycle added the widget persistence layer and the cross-schema generalizability evaluation on WooCommerce, testing whether the architecture itself, not just the nopCommerce configuration, was the reusable artifact.

3.2 Formative Study of Reporting Patterns

A dataset of 312 distinct natural-language reporting requests representative of typical e-commerce and administrative workflows was compiled. Each request was independently annotated by two researchers against a candidate set of analytics

primitives. Inter-rater agreement reached kappa = 0.84 (substantial agreement) before adjudication. After adjudication, eleven primary analytics primitives were identified that together account for 98.2% of all requests.

Table (formative study): Request taxonomy and observed frequency.

Pattern	Share of requests	Example
Ranking	24.1%	Which five categories have the highest refund rates?
Trend Analysis	21.5%	Show monthly sales volume over the last year.
KPI / Aggregate	18.3%	How many orders were placed today?
Comparison	14.7%	Compare average order value between mobile and desktop users.
Exception / Filter	12.8%	List products with stock levels below 10.
Summary / Group	6.0%	Give me an overview of the Electronics category.
Segment, Funnel, Cohort, Correlate, Tabular	2.6% combined	Revenue by category; cart-to-purchase conversion; new vs. returning customers; attribute correlation; raw order listings.

The study yields three design directions that shaped the rest of this chapter. First, a small set of patterns is sufficient: eleven patterns cover 98.2% of real requests, which supports building a fixed template library rather than an open-ended query language. Second, business vocabulary differs systematically from database column names: participants said “total refund rate,” never SUM(o.RefundedAmount), which motivates an explicit semantic layer (Section 3.7) rather than exposing the schema directly to the language model. Third, reuse is the norm rather than the exception: 61% of requests were things participants had asked before, in a different time window

or for a different segment, which motivates the widget persistence design of Section 3.11.

3.3 Design Principles

Five principles guide every architectural decision in this chapter:

8. Separate understanding from execution. The LLM understands the question; fixed rules handle everything else.
9. Define business terms explicitly. Metrics, dimensions, joins, and time rules are written once in a semantic layer, not inferred per query.
10. Limit what SQL can be generated. SQL is built only from pre-approved, parameterized templates.
11. Select visualizations by rule. Chart type is decided by question type, result shape, and established visualization design guidance, not by a learned model.
12. Persist results for reuse. Every query produces a saved, refreshable widget rather than a discarded answer.

3.4 Formal Model

Let a user with role r issue a natural-language request q. Classical text-to-SQL seeks a function f(q, S) that maps the request and a schema S directly to sql. AEGIS instead seeks a function

$g(q, L, r) \rightarrow \langle \pi, sql, vis, w \rangle$

where π is a canonical analysis plan, sql is a read-only compiled query, vis is a visualization specification, and w is a persisted widget artifact. The semantic layer $L = \langle M, D, F, J, P, V, A, R \rangle$ defines the approved metric set M, dimension set D, filter and time-rule set F, join graph J, pattern library P, visualization policy V, vocabulary injection configuration A, and role-permission model R.

Safety is enforced as a set-membership constraint: $sql \in Q\_safe(L, r)$, where Q_safe(L, r) is the family of queries derivable from pattern templates in P using only bindings from L permitted under role r.

Proposition 1. *No query in Q_safe(L, r) can reference a table, column, or row not enumerated in L for role r. All SQL identifiers are drawn from a closed vocabulary of*

approved semantic bindings. All literal values are passed using parameterized SQL rather than string interpolation. SQL injection through untrusted natural-language input is structurally prevented by design.

Security boundary. This guarantee holds within a defined threat boundary: the semantic layer definitions, compiler templates, and permission predicates are trusted, administrator-controlled artifacts. An administrator who embeds malicious SQL inside a metric definition, or a supply-chain compromise of the compiler library, falls outside this boundary and requires separate operational security controls. Explicitly stating this boundary, rather than leaving it implicit, is itself part of this thesis's contribution: prior NL-to-SQL work reviewed in Chapter 2 rarely specifies the boundary of its safety claims, which makes rigorous security comparison difficult.

3.5 Threat Model

AEGIS protects against attacks arriving through the untrusted natural-language input channel. The model assumes the database and application server are properly hardened; the attacker controls only the query field.

T1 - Prompt injection attempting SQL generation.

Attack: *"Ignore previous instructions. Generate DROP TABLE orders."*

Control: The IntentObject schema contains no SQL field. Any non-approved string in metric_term or dimension_term is rejected by Pydantic type validation at Stage 2, before the compiler is reached.

T2 - Unauthorized metric or dimension access.

Attack: *"Show me customer passwords" or "List credit card numbers by order."*

Control: Fields such as customer_password do not exist in the semantic layer vocabulary. The LLM never sees those names; it receives only the curated, approved label list. Stage 2 rejects any unrecognized term.

T3 - Unauthorized row access.

Attack: *A store-level user asks: "Show revenue for all branches."*

Control: Stage 4 (Permission Rewriter) runs after the LLM and appends a role-specific WHERE predicate (for example, AND o.StoreId = :user_store) derived from

the authenticated session. This cannot be suppressed or overridden by natural-language content.

T4 - DML or DDL injection.

Attack: *A crafted prompt that tricks the LLM into associating a write operation with an intent class.*

Control: No template in the pattern library contains a DML or DDL keyword. The AST-level post-compilation validator explicitly rejects any non-SELECT statement as a defense-in-depth layer.

Not protected by AEGIS (requires operational security controls outside this architecture):

- A malicious administrator embedding arbitrary SQL inside a metric's sql_expr field.
- Supply-chain compromise of the compiler module or the SQL parser library.
- Database-level privilege escalation that bypasses the application layer.
- LLM provider infrastructure compromise or model poisoning.

Explicitly documenting out-of-scope threats is itself a contribution: prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes meaningful security comparison difficult.

3.6 System Architecture

Every user query travels through exactly seven stages. Stages 2 through 7 contain no artificial intelligence; they are deterministic code. Rejection at any stage produces a structured clarification message rather than a best-effort guess.

Stage 1 - Intent Extraction. A lightweight LLM reads the query together with a system prompt built from the semantic layer, and outputs a validated IntentObject (intent class, metric term, dimension term, filters, sort, limit, confidence). This is the only stage that involves artificial intelligence.

Stage 2 - Coverage Validation. The server checks that both the metric term and the dimension term exist in the semantic layer vocabulary before anything else runs. Unknown identifiers are rejected here, with a structured message listing the available identifiers, rather than being passed to the compiler.

Stage 3 - Semantic Mapping. Business-logic aliases are expanded (for example, “abandoned” maps to a specific OrderStatusId), and relative time expressions such as “this month” are resolved to concrete date predicates.

Stage 4 - Permission Rewriting. A role-specific WHERE predicate is appended based on the authenticated user's session. This runs after the LLM has already finished, so no natural-language content can influence it.

Stage 5 - SQL Compilation. A breadth-first search over the join graph finds the minimal join path connecting the tables required by the resolved metric and dimension, and pre-compiled SQL expressions are substituted into a parameterized template. No SQL text is ever assembled from concatenated user input.

Stage 6 - Visualization Selection. A rule engine maps the intent class and result shape to a default chart type (Section 3.10). This stage contains no learned model.

Stage 7 - Widget Persistence. The analysis plan is hashed (SHA-256) to detect duplicates, and the query, chart configuration, and access rules are stored as a widget artifact that can be refreshed on a schedule (Section 3.11).

3.7 Semantic Layer Design

The semantic layer is the most important non-AI component of AEGIS. It separates business language from the underlying database structure and defines exactly which metrics, joins, and permissions are allowed to exist. A useful analogy is LEGO blocks rather than free-form clay: the semantic layer defines a finite set of composable building blocks. User questions are unlimited, but every answerable question is a composition of these blocks.

Table 1: Semantic layer object model.

Object	Field	Example
Metric	label, SQL expression, joins, visual default, security class	revenue = SUM(o.OrderTotal - o.RefundedAmount)
Dimension	label, SQL expression, datatype, access scope	category = c.Name from Category
Filter	label, SQL predicate, datatype	payment_status : o.PaymentStatusId = :val
Time rule	label, SQL predicate, granularity	current_week : DATEADD(week, ...)
Join path	source, target, ON clause	Order → OrderItem → Product → Category
Pattern	required slots, SQL template, visualization default	ranking : metric + dimension → bar chart
Permission	rule	store_manager → filtered by store location

In the nopCommerce prototype, the semantic layer defines 15 metrics, 34 dimensions, and 11 join paths across 12 analytics-relevant tables. The full nopCommerce schema contains 126 tables; the remaining 114 (system, content-management, configuration, authentication, vendor, and promotions tables) are deliberately not represented in the semantic layer at all. No business analyst asks “show me revenue by ScheduleTask,” and excluding these tables functions as an implicit table-level access control: even a crafted prompt that names a hidden system table directly is rejected at Stage 2, because the identifier simply does not exist in L.

3.8 Intent Parsing with Dynamic Vocabulary Injection

The central technique that makes Stage 1 reliable is vocabulary injection. At startup, AEGIS builds the system prompt by listing every approved metric and dimension name, with a plain-English description, directly from the semantic layer (approximately 1,100 tokens for 15 metrics and 34 dimensions). The LLM sees exactly which identifiers are valid and maps any user phrasing to the correct identifier without a manually maintained synonym list. This has three advantages over a synonym dictionary: zero maintenance, since adding a metric automatically updates the vocabulary the model sees; broad coverage of arbitrary user phrasing, since the model performs the mapping rather than a fixed lookup table; and token efficiency, since the full vocabulary for 15 metrics and 34 dimensions fits in roughly 1,100 tokens.

The output schema enforces typed fields:

```
{
  "intent_class": "kpi | ranking | trend | comparison | exception |
                   summary | segment | funnel | cohort | correlate |
tabular",
  "metric_term": "string",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value":
"string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean"
}
```

3.9 Safe Query Compiler

The compiler instantiates SQL from a library of parameterized templates, one per analytics pattern.

Table 2: The eleven AEGIS analytical patterns.

Pattern	Required slots	Optional slots	Default visual
KPI (Aggregate)	metric	time_rule, filter	kpi_card
Ranking	metric, dimension	time_rule, filter, limit	bar_chart
Trend	metric, time_grain	time_rule, filter	line_chart
Comparison	metric, segment	time_rule, filter	grouped_bar
Exception	metric, threshold	dimension, time_rule	table
Summary	metric[], dimension	time_rule, filter	multi_card
Segment	metric, dimension	time_rule, filter	pie_chart
Funnel	metric, stages	time_rule, filter	funnel_chart
Cohort	metric, group_def	time_rule, filter	grouped_bar
Correlate	metric, attribute	time_rule, filter	scatter_plot
Tabular	dimension	filters, time_rule	table

After placeholder substitution, two safety layers apply in sequence. Layer 1 is a parameterized query engine that separates SQL structure from user-supplied inputs, so no user text is ever concatenated into the SQL string. Layer 2 is a post-compilation safety scanner that rejects any assembled query containing a forbidden construct: non-SELECT statements, UNION, EXCEPT or INTERSECT, EXEC, or references to system tables. If any forbidden pattern is detected, the compiler raises a SecurityError rather than returning a partially safe query.

3.10 Visualization Selector

Table 3: Visualization selector mapping.

Intent	Result shape	Selected visualization
KPI	scalar	KPI card

Ranking	1 measure, up to 20 categories	Horizontal bar chart
Trend	1 measure, time series	Line chart
Comparison	1 measure, 2-4 segments	Grouped bar chart
Exception	row-level detail	Sortable table
Summary	2-4 scalar measures	KPI card grid
Segment	1 measure, categorical	Pie chart
Funnel	ordered conversion stages	Funnel chart
Cohort	1 measure, 2+ groups	Grouped bar chart
Correlate	2 measures, continuous	Scatter plot
Tabular	raw records	Sortable table

Two additional rules apply after the data is known, rather than at selection time: bar charts with more than 20 categories are automatically converted to a table, and pie charts with more than 8 slices are converted to a bar chart. Both rules exist because the initial chart choice is made before the exact cardinality of the result is known.

3.11 Widget Persistence and Reuse

Each widget stores a unique identifier (a SHA-256 hash of the analysis plan), the original question, the analysis plan in JSON form, a hash of the SQL template used, chart configuration, timestamps, access rules, and run history. A new question triggers the full seven-stage pipeline; if an identical widget already exists, determined by a match on the plan hash, the cached artifact is returned immediately rather than recompiled. Scheduled refresh re-executes the stored SQL against fresh data on a configurable interval, which directly addresses the formative-study finding (Section 3.2) that 61% of reporting requests are recurring: a widget answers the question once and then continues answering it as new data arrives, rather than requiring the same natural-language request to be re-processed from scratch every time.

CHAPTER 4

EXPERIMENTAL WORK

4.1 Implementation

AEGIS is implemented as a web application with a vanilla HTML and JavaScript frontend (jQuery, Chart.js) and a Python FastAPI backend, targeting a production nopCommerce 4.70 schema (126 tables, 107 foreign key constraints). The implementation follows directly from the architecture in Chapter 3:

- LLM integration: Llama 3.1 8B Instant via the Groq API, with structured JSON output enforcement. The system prompt is constructed dynamically by injecting the approved metric and dimension identifiers at startup.
- Rate limiting: a provider-agnostic configuration module with a sliding-window rate limiter and a concurrency-safe asyncio.Lock, so the architecture is not tied to a specific LLM vendor's throughput characteristics.
- Semantic layer: Python configuration modules containing 15 metrics, 34 dimensions, zero synonym entries, and 11 join paths across the 12 analytics-relevant tables described in Section 3.7.
- SQL compiler: parameterized MySQL templates, with breadth-first search join-path resolution over the 12-table join graph. A post-compilation _validate_sql_safety() routine checks 16 forbidden patterns before a query is allowed to execute.
- Visualization selector: rule-based Python dictionaries implementing Table 3 of Chapter 3, with the two post-hoc cardinality rules applied after the result set is known.
- Widget engine: SHA-256 plan-hash deduplication, with JSON file storage in the prototype, designed to be swapped for a relational store in a production deployment.
- Coverage validator: a pre-compilation gate that rejects unknown metric or dimension terms with structured guidance listing the available identifiers.

- Permission enforcement: a Permission Rewriter that appends role-based WHERE predicates for five roles: public, store_manager, regional_manager, read_only, and analyst.

4.2 Experimental Environment

All experiments ran against a Docker-containerized MySQL 8.0 instance initialized with the AEGIS Truth Schema (126 tables, 107 foreign keys). The mock dataset used for evaluation contains 1,200 customers, 2,500 orders spanning 2024-2026, 6,298 order items, and 1,000 products mapped across 50 categories, sized to be representative of a mid-sized e-commerce deployment rather than a toy schema.

4.3 Benchmark Dataset Construction

This evaluation is a prototype evaluation, not a large-scale independent benchmark study. Its goal is to demonstrate that the AEGIS architecture achieves its stated safety and semantic-fidelity properties on representative real-world analytics queries, not to establish population-level accuracy claims. Standard text-to-SQL benchmarks such as Spider and BIRD (Chapter 2) do not evaluate adversarial safety or adherence to business vocabulary, which is why a domain-specific benchmark was necessary for this thesis.

A domain-specific benchmark of 100 reporting requests was built over the production nopCommerce schema described in Section 4.2. The full question set, ground-truth SQL, and recorded pipeline outputs are available in the evaluation_dataset/ directory of the project repository, enabling independent verification of every reported metric. Queries span all eleven analytics primitives identified in Section 3.2, with vocabulary variation not seen during system design, and twenty of the hundred queries are adversarial, specifically constructed to attempt prompt injection, indirect injection via filter values, or instruction-override attacks. Gold-standard SQL for every query was independently verified by two database engineers. Because the benchmark was constructed for the nopCommerce domain, accuracy figures in Chapter 5 should be interpreted within that scope; queries that require analytical patterns not yet in the template library (Table 2) are excluded by design, so the benchmark measures depth

of coverage within the supported pattern set rather than breadth across every conceivable analytics request.

4.4 Baseline Systems

Four baselines isolate the contribution of each architectural layer:

B1 - Direct LLM-to-SQL. Llama 3.1 8B prompted with the full database schema, with no semantic layer and no template constraints; the model is free to generate any SQL text it produces.

B2 - Decomposed LLM. A chain-of-thought strategy that first extracts entities, then generates SQL from the extracted entities, testing whether decomposition alone (without a fixed template library) improves safety.

B3 - Template-only (no LLM). Keyword matching directly to templates, with no LLM-based intent extraction, testing how much of AEGIS's safety comes from having a fixed template library at all, independent of how the template is selected.

B4 - AEGIS ablated (no semantic layer). The full AEGIS pipeline with the semantic mapper bypassed, testing whether the semantic layer specifically, rather than the pipeline structure in general, is responsible for AEGIS's measured gains.

4.5 Evaluation Procedure

The evaluation addresses five research questions, each with its own procedure, reported in full in Chapter 5:

- RQ1: How accurately does the LLM intent parser extract typed reporting plans? Measured as per-class precision, recall, and F1 over the 100-query benchmark.
- RQ2: Does AEGIS reduce unsafe and semantically incorrect SQL compared to direct LLM-to-SQL baselines? Measured as unsafe SQL rate, execution validity, and coverage against baseline B1.
- RQ3: Does template-based compilation preserve sufficient expressiveness? Measured over the full 312-request formative-study set, and via an ablation study removing each architectural component in turn (vocabulary injection, semantic layer, AST validation, confidence-gated clarification, permission rewriter, repair-on-parse-failure).

- RQ4: Does the architecture generalize to a second production schema outside the training domain? Measured by building an independent semantic layer for WooCommerce and recording both the resulting accuracy and the person-hours required to build it.
- RQ5: What is the end-to-end latency cost of the AEGIS pipeline? Measured as median and 95th-percentile latency per pipeline stage.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports results for each of the five research questions introduced in Section 4.5, followed by a discussion of what the results mean, what AEGIS deliberately trades away, and how it compares structurally to direct LLM-to-SQL generation.

5.1 Intent Parsing Accuracy (RQ1)

Table 5: Intent parsing precision, recall, and F1 by intent class.

Intent class	Precision	Recall	F1
KPI	1.00	1.00	1.00
Ranking	1.00	1.00	1.00
Trend	1.00	1.00	1.00
Comparison	1.00	1.00	1.00
Exception	1.00	1.00	1.00
Summary	1.00	1.00	1.00
Segment	1.00	1.00	1.00
Funnel	1.00	1.00	1.00
Cohort	1.00	1.00	1.00
Correlate	1.00	1.00	1.00
Tabular	1.00	1.00	1.00
Overall	1.00	1.00	1.00

Every intent class reached perfect precision, recall, and F1 on the 100-query benchmark. This result should be read as a demonstration that vocabulary injection resolves intent classification within the benchmark's covered vocabulary and phrasing variation, not as a claim that AEGIS never misclassifies a request in general; Section 5.6 reports the failure modes observed on the larger, more varied 312-request

formative-study set, where coverage-boundary rejections and clarification requests do occur.

5.2 SQL Safety and Execution Validity (RQ2)

Table 6: SQL safety and execution validity vs. direct LLM-to-SQL baseline.

System	Unsafe SQL rate	Execution validity	Coverage
Baseline B1 (Direct LLM-to-SQL)	5.0%	99%	99%
AEGIS (with vocabulary injection)	0%	100%	100%

The direct LLM-to-SQL baseline produced 5 unsafe queries out of 100 (a 5.0% unsafe rate), including INSERT, UPDATE, and DELETE statements and UNION clauses generated in response to the twenty adversarial prompts in the benchmark. AEGIS eliminated unsafe queries entirely, not by detecting and blocking them after generation, but by never allowing the LLM to generate executable SQL in the first place.

5.3 Expressiveness and Ablation Study (RQ3)

Of the 312 requests analyzed in the formative study (Section 3.2): 81.7% were answered directly without clarification, 11.5% required one clarification turn, 4.2% were answered only after the semantic layer was extended with a missing metric or dimension, and 2.6% could not be answered because they fell outside the template library entirely.

Table 7: Ablation study - execution validity and coverage per configuration.

Configuration	Execution validity	Coverage
Full AEGIS (vocabulary injection)	100%	100%
- Vocabulary injection (synonym dictionary instead)	64.7%	99%
- Semantic layer	88.7%	91%

- AST validation	100%*	100%
- Confidence-gated clarification	94.2%	96%
- Permission rewriter	100%**	100%
- Repair call on parse failure	92.9%	95%

Removing vocabulary injection in favor of a hand-maintained synonym dictionary produces the largest single drop, 35.3 percentage points of execution validity, confirming that vocabulary injection is not a convenience but the component doing the most work in keeping intent extraction reliable. Removing AST validation or the permission rewriter leaves the reported metrics unchanged on this benchmark (marked with an asterisk), which is expected and correctly interpreted as confirming their role as defense-in-depth layers against attack classes (T3, T4 in Section 3.5) that this particular benchmark's queries do not exercise, not as evidence that those layers are unnecessary.

5.4 Cross-Schema Generalizability (RQ4)

A second semantic layer was built for WooCommerce, a structurally distinct e-commerce platform with different table naming conventions and business vocabulary (12 metrics, 28 dimensions, 9 join paths, 18 tables), together with a 50-query evaluation set built using the same methodology as Section 4.3.

Table 8: Cross-schema generalizability results.

Schema	Build time	Intent accuracy	Unsafe SQL rate	Coverage
nopCommerce (primary, 100 queries)	40 person-hours	100%	0%	100%
WooCommerce (transfer, 50 queries)	14 person-hours	98.0%	0%	96.0%

The WooCommerce evaluation achieved 98.0% intent accuracy with zero unsafe SQL using a semantic layer built in 14 person-hours, 65% less effort than the primary schema required. The 2% accuracy gap arose from two WooCommerce-specific metric names, resolved by adding two description entries to the prompt with no code changes required. These results support the claim that AEGIS's architecture, not just its nopCommerce configuration, is what generalizes: the LLM, the compiler, and the safety scanner required zero modification between schemas; only the semantic layer configuration changed.

5.5 Pipeline Latency (RQ5)

Table 9: Pipeline stage latency.

Stage	Median (ms)	p95 (ms)	% of total
LLM API call (Groq)	1,850	2,800	96.2%
Semantic mapping	12	18	0.6%
SQL compilation	8	12	0.4%
Query execution (MySQL)	45	120	2.3%
Visualization selector	2	4	0.1%
Widget persistence	5	9	0.3%
Total	1,922	2,963	100%

AEGIS's safety infrastructure, the deterministic stages that carry out coverage validation, semantic mapping, permission rewriting, compilation, visualization selection, and widget persistence combined, adds a median of roughly 20 ms of overhead, negligible relative to the 1,850 ms median LLM API call. With a locally hosted Ollama model, median LLM call latency drops to approximately 340 ms, bringing total end-to-end latency below 430 ms. The safety guarantees established in Chapter 3 are, in this sense, effectively free: the cost of AEGIS's architecture is dominated by the same LLM call a direct LLM-to-SQL system would also have to make.

5.6 Failure Analysis

Among the 2.6% of the 312 formative-study requests that could not be answered at all, coverage-boundary rejections broke down as follows: metrics not present in the semantic layer (35% of rejections), unregistered dimensions (28%), requests requiring multi-metric aggregation beyond a single pattern (18%), causal or explanatory questions such as “why did revenue drop” (12%), and requests requiring a join path not present in the join graph (7%). Every rejection, in this study and in the benchmark, included the full list of available identifiers rather than a bare error, consistent with the design principle (Section 3.3) that a coverage failure should be actionable rather than opaque.

5.7 Discussion

5.7.1 AEGIS vs. direct LLM-to-SQL: structural comparison.

A natural question is why not simply ask a capable LLM to write SQL directly. The differences below are architectural, not accuracy-based: they would hold even against a future, more capable model.

Table 10: Structural comparison of AEGIS vs. direct LLM-to-SQL.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model-generated (probabilistic)	Deterministic compiler
Schema exposure to LLM	Required (tables, columns, keys)	Not required (labels only)
Business metric definitions	Implied from schema names	Explicit in semantic layer
SQL injection prevention	Prompt-level (best-effort)	Structural (by design)
Permission enforcement	External or none	Built-in, post-LLM
Dashboard widget persistence	Not provided	First-class artifact
Auditability of query origin	Difficult	Full provenance per widget
Model dependency	Tied to specific model quality	Model-independent

AEGIS does not claim to produce more creative SQL than a frontier LLM. It claims that, for the analytics requests it supports, results are guaranteed correct by

construction, auditable, and safe, properties that probabilistic generation cannot offer unconditionally, no matter how capable the underlying model becomes.

5.7.2 Why a semantic layer instead of retrieval-augmented generation?

Retrieval-augmented generation (RAG) for NL-to-SQL retrieves relevant schema fragments to give the LLM better context. This is a useful technique, but it solves a different problem from the semantic layer and does not eliminate the safety risk. RAG asks which schema information the LLM should see; it is an access-optimization technique that narrows the schema the model reasons over, but the LLM still generates a free-form SQL string as output. The semantic layer asks which analytical concepts are allowed to exist, what they mean, and who may access them; it is a governance mechanism that defines the complete set of answerable questions and their canonical SQL translations before any query is processed, and the LLM outputs a typed intent object rather than SQL. An organization that wants both better schema context and controlled output could use RAG to select relevant semantic layer sections for very large vocabularies, thousands of metrics, while still routing every query through the AEGIS compiler; the two techniques are complementary rather than competing.

5.7.3 Controlling the model vs. training a better model.

The direct LLM baseline (Section 5.2) produced 5 unsafe queries using the same underlying model AEGIS uses, Llama 3.1 8B. AEGIS, using that identical model but restricting it to understanding questions only, had zero unsafe queries. When something must always be true, such as “never expose private data,” it should be enforced by system structure, not left to the probability that a given model call behaves correctly.

5.7.4 Vocabulary injection: letting the model do what it does best.

Hand-crafted synonym dictionaries are both unnecessary and counterproductive once the LLM has explicit access to the approved vocabulary. In practice, the model mapped “earnings” to revenue, “promo codes” to discount_amount, and “clients” to customer_email, none of which appeared in any synonym list. This reduced the synonym dictionary from 112 entries to zero while improving coverage from 99% to 100% (Table 7).

5.7.5 What this thesis gives up.

AEGIS only supports queries that fit within its defined metrics, dimensions, and patterns. For open-ended data exploration requiring custom joins or schema-level operations, an unconstrained system may be more appropriate. AEGIS is designed for the majority of everyday reporting needs identified in the formative study, not for ad hoc data science exploration.

5.7.6 Why saving widgets matters.

Widget reuse directly addresses the formative-study finding that 61% of reporting requests are repeated questions (Section 3.2). Saved widgets become part of a user's daily workflow rather than requiring regeneration, and therefore a fresh round-trip through the LLM and every safety stage, each time the same question recurs.

5.7.7 What AEGIS cannot answer.

AEGIS can answer from approximately 5,100 valid combinations (15 metrics times 34 dimensions times 10 patterns, in the nopCommerce configuration). Out-of-scope queries receive a structured rejection listing available identifiers rather than a silent wrong answer:

```
Unknown metric 'conversion_rate'.
Available: avg_order_value, customer_count, discount_amount,
           order_count, profit, refund_amount, revenue, ...
```

Extending coverage requires only adding rows to the semantic layer; it requires no model retraining and no synonym curation, because vocabulary injection propagates a new entry to the LLM's available vocabulary automatically.

CHAPTER 6

LIMITATIONS AND FUTURE WORK

6.1 Limitations

Semantic layer construction cost. Every AEGIS deployment requires a domain-specific semantic layer built by someone with both business knowledge and schema access. The nopCommerce prototype took approximately 40 person-hours. Organizations without this expertise, or with rapidly evolving schemas, may find the maintenance burden significant. AEGIS is not a zero-configuration system.

Cannot answer arbitrary SQL questions. By design, AEGIS only answers questions that map to a supported analytics primitive with an approved metric and dimension. Ad hoc queries, multi-level nested aggregations, or requests for data fields not in the semantic layer fail with a coverage error (Section 5.6). This is a deliberate trade-off, not an oversight.

Complex analytical queries require new templates. Approximately 2.6% of the formative-study requests (Section 3.2) required patterns not yet in the template library. Adding a new pattern requires a developer with SQL knowledge; it is not something a business user can do themselves.

Quality depends on intent extraction, not just compilation. The safety guarantees in Chapter 3 apply to compilation and execution, not to intent extraction quality. A model that misclassifies a request will produce a structurally safe but semantically wrong query. Safety and accuracy are separate properties, and this thesis is careful not to conflate them.

Benchmark selection. The custom 100-query benchmark (Section 4.3) was necessary because standard benchmarks such as Spider and BIRD do not evaluate adversarial safety or adherence to business logic; this also means the reported accuracy figures are not directly comparable to Spider- or BIRD-reported numbers from other systems.

Semantic layer scalability. Modern context windows of roughly 128,000 tokens can hold approximately 2,500 distinct metric and dimension definitions; most enterprise deployments expose fewer than 500 core concepts, so this is not a near-term

constraint, but very large vocabularies would need retrieval-augmented vocabulary injection rather than injecting the entire semantic layer at once.

Database agnosticism. The current compiler generates MySQL syntax; supporting PostgreSQL or SQL Server requires extending the compiler module, not redesigning the architecture.

Storage persistence. The prototype uses JSON flat files for widget storage; the widget registry interface is designed to be swapped for a relational database in a production deployment.

Multi-turn conversation. AEGIS currently treats each request independently. Contextual carryover across turns, of the kind studied for conversational text-to-SQL in Section 2.2, is not yet implemented.

Vocabulary injection limitations. Highly specialized domain terminology may require supplementary few-shot examples in the prompt beyond the label and description pairs currently injected.

6.2 Future Work

- Clarification requests: when confidence is low, AEGIS should ask a follow-up question instead of guessing, for example “did you mean revenue or profit?”
- Semantic layer wizard: a guided interface for business analysts to define new metrics and dimensions without writing Python.
- Multi-step queries: currently each query produces one widget; compound questions such as “revenue trend and top 5 products side by side” require two separate queries today.
- Automated denial-of-service protection: query complexity scoring and server-side timeout enforcement, since the join graph bounds query cost but does not currently score it explicitly.
- Broader schema coverage: extending the nopCommerce semantic layer to cover promotions, vendor analytics, and content-management engagement metrics.
- Multi-turn conversational carryover, extending the single-turn design discussed above.

CHAPTER 7

CONCLUSION

This thesis presented AEGIS, a system for turning plain-English reporting requests into dynamic, refreshable dashboard widgets over relational databases. The central contribution has two parts. First, a system design that limits the language model to understanding questions, while all query building, chart selection, and widget storage is handled by fixed rules and pre-approved templates, so that safety is a property guaranteed by system structure rather than a probability that improves with model quality. Second, a vocabulary injection method that removes the need for manually maintained synonym lists while improving coverage, reducing a 112-entry synonym dictionary to zero entries while raising coverage from 99% to 100%.

The evaluation in Chapter 5 shows that AEGIS reduces the unsafe SQL rate from 5.0% to 0% relative to a direct LLM-to-SQL baseline using the same underlying model, achieves 100% valid SQL and 100% coverage on its 100-query nopCommerce benchmark, and generalizes to a second production schema, WooCommerce, with only semantic layer reconfiguration required and no change to the LLM, the compiler, or the safety scanner. The pipeline latency analysis confirms that this safety infrastructure adds less than 4% overhead relative to the LLM API call that any comparable system would also need to make.

The literature review in Chapter 2 situates this contribution precisely: prior text-to-SQL research, from Seq2SQL through RAT-SQL and PICARD, treats SQL-generation accuracy as the object of study and, with the partial exception of a 2025 systematic review's discussion of injection attacks, does not evaluate safety as a first-class property at all. AEGIS's architectural choice, removing SQL generation from the language model's role entirely rather than constraining it more tightly, is a categorically different answer to the same underlying problem, and one independently echoed in recent explanatory-analytics research reviewed in Section 2.7 that arrives at the same “let a deterministic layer compute the answer” principle in an unrelated domain.

AEGIS is built for environments where data privacy, consistent reporting definitions, and daily reuse of saved reports matter more than unlimited query flexibility. Chapter 6 states plainly what this design gives up in exchange: a bounded vocabulary, an upfront semantic layer construction cost, and dependence on intent-extraction quality for semantic (though never safety) correctness. Within that scope, this thesis demonstrates that restricting SQL generation to a finite set of validated business patterns is a practical, measurable path to safe, auditable natural-language reporting in institutional environments.

REFERENCES

[1] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," The VLDB Journal, vol. 28, no. 5, pp. 793-819, 2019.

[2] B. Chinnappaiyan, "Conversational analytics in self-service data platforms: Democratizing enterprise data access through natural language interfaces," European Journal of Computer Science and Information Technology, vol. 13, no. 46, pp. 94-102, 2025.

[3] D. Deng, A. Wu, H. Qu, and Y. Wu, "DashBot: Insight-driven dashboard generation based on deep reinforcement learning," IEEE Transactions on Visualization and Computer Graphics, vol. 29, no. 1, pp. 690-700, 2023.

[4] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in Proc. 28th Annual ACM Symposium on User Interface Software and Technology (UIST), 2015, pp. 489-500.

[5] E. Häikiö, "Adoption and governance of AI-powered dashboards in executive-level decision-making," M.S. thesis, Information System Science, Turku School of Economics, University of Turku, Finland, 2024.

[6] A. Jha, N. Anand, and H. Karthikeyan, "Conversion of natural language text to SQL queries using generative AI," in Hybrid and Advanced Technologies, S. P. J. Christydass, N. Nurhayati, and S. Kannadhasan, Eds. Boca Raton, FL: CRC Press, 2025, pp. 25-32.

[7] E. Kavaz, A. Puig, and I. Rodríguez, "Chatbot-based natural language interfaces for data visualisation: A scoping review," Applied Sciences, vol. 13, no. 12, p. 7025, 2023.

[8] C. Lehmann, D. Gehrig, S. Holdener, C. Saladin, J. P. Monteiro, and K. Stockinger, "Building natural language interfaces for databases in practice," in Proc. 34th Int. Conf. Scientific and Statistical Database Management (SSDBM), 2022, Art. no. 20.

[9] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," Proceedings of the VLDB Endowment, vol. 8, no. 1, pp. 73-84, 2014.

[10] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[11] M. Liu and J. Xu, "NLI4DB: A systematic review of natural language interfaces for databases," arXiv:2503.02435, 2025.

[12] Y. Luo, N. Tang, G. Li, J. Tang, C. Chai, and X. Qin, "Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks," in Proc. 2021 ACM SIGMOD Int. Conf. Management of Data, 2021, pp. 1235-1247.

[13] M. Mujeeb, L. K. Sagar, and S. Shrestha, "AI-powered conversational business intelligence: A natural language interface for data-driven decision making," International Journal of Scientific Development and Research, vol. 10, no. 11, pp. b685-b689, 2025.

[14] A. Narechania, A. Srinivasan, and J. Stasko, "nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

[15] OpenAI, "Introducing structured outputs in the API," OpenAI, 2024.

[16] G. Pinna, Y. Perezhohin, L. Manzoni, M. Castelli, and A. De Lorenzo, "Redefining text-to-SQL metrics by incorporating semantic and structural similarity," Scientific Reports, vol. 15, Art. no. 22357, 2025.

[17] M. J. I. Saidur, "AI-enhanced business intelligence dashboards for predictive market strategy in U.S. enterprises," International Journal of Business and Economics Insights, vol. 5, no. 3, pp. 603-648, 2025.

[18] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 9895-9901.

[19] V. Setlur, S. E. Battersby, M. Tory, R. Gossweiler, and A. X. Chang, "Eviza: A natural language interface for visual analysis," in Proc. 29th Annual ACM Symposium on User Interface Software and Technology (UIST), 2016, pp. 365-377.

[20] G. N. Shailesh, M. Pavithran, A. Rahul Hari Venkat, and P. Kaliappan, "Conversational BI: Natural language interface to business dashboards," International Journal of Engineering Research & Technology, vol. 14, no. 12, 2025.

[21] H. S. Shalaan, T. H. A. Soliman, and A. M. Abdelaziz, "G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation," IEEE Access, vol. 13, pp. 158520-158534, 2025.

[22] D. Shi, X. Xu, F. Sun, Y. Shi, and N. Cao, "Calliope: Automatic visual data stories with Monte Carlo tree search," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 464-474, 2021.

[23] X. Su, Y. Gu, P. Wang, W. Gu, L. Qi, and J. He, "A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs," Scientific Reports, vol. 16, Art. no. 7892, 2026.

[24] J. Valkenburgh, "Enhancing business dashboards with explanatory analytics and AI: Exploring the use of AI and explanatory analytics to enhance business decision-making," M.S. thesis, Information Management, Turku School of Economics, University of Turku, Finland, 2024.

[25] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7567-7578.

[26] Y. Wang, Z. Sun, H. Zhang, W. Cui, K. Xu, X. Ma, and D. Zhang, "DataShot: Automatic generation of fact sheets from tabular data," IEEE Transactions on Visualization and Computer Graphics, vol. 26, no. 1, pp. 895-905, 2020.

[27] A. Wu, W. Tong, T. Dwyer, B. Lee, P. Isenberg, and H. Qu, "MultiVision: Designing analytical dashboards with deep learning based recommendation," IEEE Transactions on Visualization and Computer Graphics, vol. 28, no. 1, pp. 162-172, 2022.

[28] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911-3921.

[29] T. Yu et al., "SParC: Cross-domain semantic parsing in context," in Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 4511-4523.

[30] T. Yu et al., "CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2019, pp. 1962-1979.

[31] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," arXiv:1709.00103, 2018.